

APPENDIX FOR “PLOTTERY: A CODE-INTEGRATED GUI FOR MATPLOTLIB”

Below are descriptions of the AI prompt and several case studies of creating plots for the paper. For a video walkthrough and the user study materials, see the replication package at [plottery.org](#) [53].

A. AI Code-Generation Prompt

The AI prompt in TypeScript is:

```
const prompt = `
I am writing matplotlib code in the last cell of the following
notebook:

\`\`\`
${notebook_code}
\`\`\`

Modify the code of Cell ${last_cell_no} to ${user_prompt}

Note: If I asked to create a function for a subplot and you need to
pass in an ax argument, give that parameter a type annotation
of mpl.axes._axes.Axes, but _only_ if there is a custom
function for the subplot.

Note: If I asked above to add more subplots, use the following form:

\`\`\`
fig = plt.figure(...)
ax1 = fig.add_subplot(...)
ax2 = fig.add_subplot(...)
...
\`\`\`

but _only_ if I asked to add more subplots.

Return only the new code of Cell ${last_cell_no}`
```

The `notebook_code` is the code cells in the notebook (Markdown cells are excluded) joined into one long string, up to and including the current cell (later cells are not included). Each cell is prefixed with a comment like `### Cell 1 ###` indicating the cell number. The note about adding annotating the `ax` parameter was added after all the user studies and case studies were completed, it helps MyPy infer helpful types within the function.

B. Case Studies

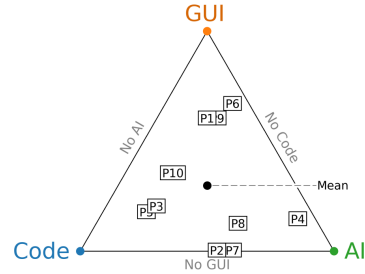
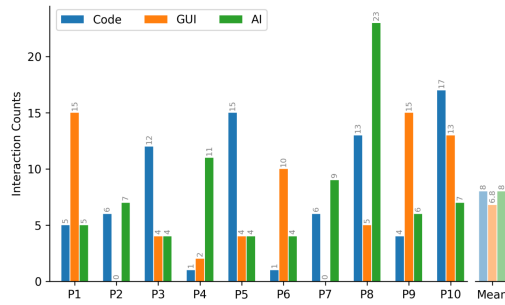
The user study tasks replicated existing plots. Here we present three brief case studies of using PLOTTERY to make real world plots *from scratch*: namely, the plots in drafts of the paper. We focused on, but did not limit ourselves to, GUI interactions.

Case Study 1 - Grouped Bar Charts. We used PLOTTERY to create the grouped bar charts shown in Fig. 16 and Fig. 17, similar to the “Group bar chart with labels” example from the Matplotlib documentation.¹ Grouped bar charts are difficult to implement in Matplotlib—there’s no built-in function for it so the user must programmatically position and size each bar series manually. In PLOTTERY, this can be done graphically. The authoring is similar to the overview example in Sec. III, with few notable differences:

- 1) We create three overlapping bar series directly manipulate them on the plot to not only resize their widths but also translate them to be next to each other. The translation

¹https://matplotlib.org/stable/gallery/lines_bars_and_markers/barchart.html

A Part 1: Interaction Counts



B Part 2: Interaction Counts

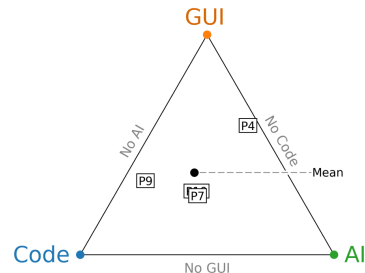
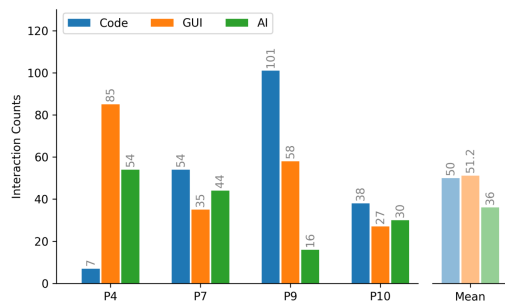
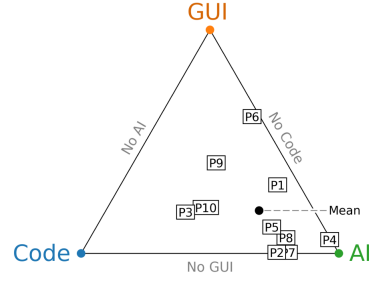
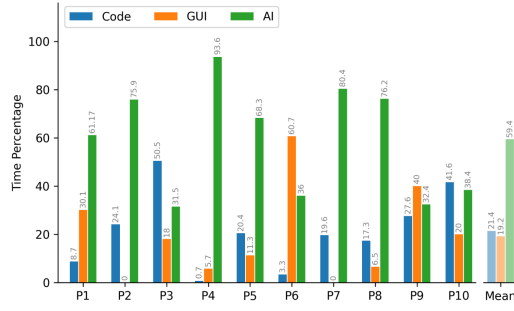


Fig. 16: Interaction counts when participants could use code, GUI, and AI freely: **A** PART 1 task 3 and **B** PART 2 tasks.

A Part 1: Time Percentage



B Part 2: Time Percentage

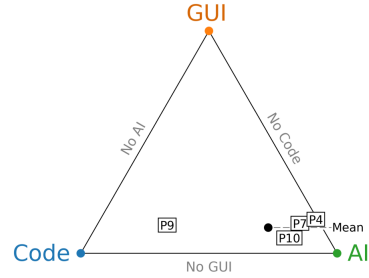
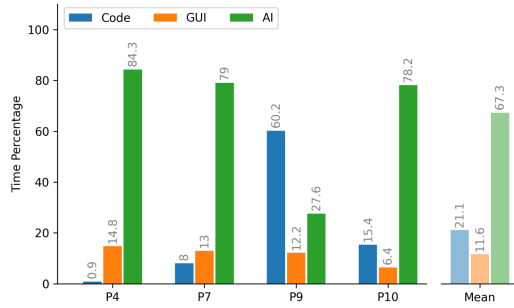


Fig. 17: As in Fig. 16, but percentage of *interaction time* in each modality for **A** PART 1 task 3 and **B** PART 2 tasks. For both parts, AI occupies a majority of interaction time and GUI the least.

requires that their x positions first be the the NumPy array `np.arange(len(participants))` so that adding a single number during the mouse-drag (see Sec. IV) is a valid operation, because adding a number to an ordinary Python list is an error.

- 2) We similarly position the legend simply by dragging it to the desired location.
- 3) We abstracted the final plot into a function (possible via AI or code edits).

In Summary: PLOTTERY allows fine-tune placement of on-plot elements by directly dragging and resizing, while the code facilitates later reusability through traditional coding abstractions, namely functions. This plot was particularly suited to PLOTTERY because it involved common Matplotlib functions with no data transformation.

Case Study 2 - Ternary Plot. PLOTTERY also assisted us in creating the triangle plots in Figures 16 and 17. Again, this kind of plot is not built in to Matplotlib, so it requires a lot of manual placement. The code for computing the xy coordinates for the data points on the triangle was written manually. We used PLOTTERY to check this code for correctness by constructing an ad-hoc interactive plot, shown in Fig. 18. We set up this plot so we could drag the dot around the plot and see live updates of the distance to the edges, shown in text next to each edge, as well as a “Loss” which should be 0 when the dot is at the right location. Once satisfied these computations were correct—there were mistakes on the first

try—we computed the xy coordinates for all participants and created the plot.

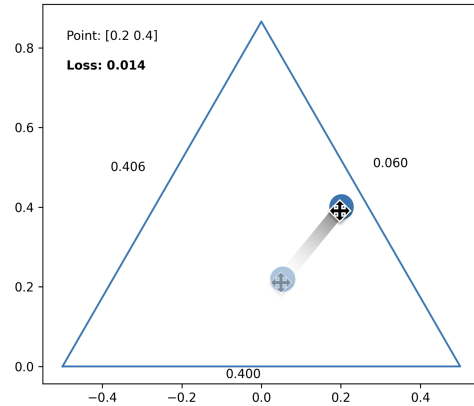


Fig. 18: An ad-hoc interactive plot for checking the calculation of the distance to the lines and the loss from the correct location.

The initial plot was created without consulting the Matplotlib documentation. For example, the default triangle corners were slightly rounded instead of sharp. Rather than searching online, we toggled a few properties to find that `solid_joinstyle` changed the corners, and we didn’t need to look up the valid values because the API’s type annotations produced the suggestions shown in Fig. 19. The live preview

instantly revealed that 'miter' produced sharp corners . The Properties panel also helped style the text, but we had to repeat the edits three times for the labels around the triangles—first class support for linking properties via a shared variable would have helped. We used AI to lock the aspect ratio of the x and y scales so the triangle was correctly equilateral, and with manual code edits we abstracted the plot into a function and extracted `border_color` into a shared variable.



Fig. 19: Type-derived suggestions for `solid_joinstyle`.

After adding more participants to our study, we revisited the plot to make the participant numbers more legible. Most notably, the AI helped us add the `bbox` argument to `ax.text`, which we might never have found by searching through documentation. Unfortunately, PLOTTERY's Properties panel could not adjust details of the `bbox` because it is a yet-unsupported `dict` object. Other GUI operations continued to work, for example when some numbers overlapped the existing text annotations ("No GUI" on the code-AI edge) we were immediately able to drag the text annotation out of the way via the GUI.

The final versions shown in Fig. 14 and Fig. 15 in the main paper were produced by a final round of re-styling. This was done without AI, mostly by GUI tweaking of visual properties and dragging items around. To get white outlines around the circle marks, code edits were used to duplicate the scatterplot and make a larger white version underneath. Code edits were also used to work out the math for the "knee" corner in the dotted "Mean" line on two of the plots. This offset amount for the corner was added as another parameter to the whole ternary plot drawing function, so the final amount of offset was tweaked using the GUI by selecting the appropriate drawing function call and dragging the numeric slider in the properties panel for the offset.

In Summary: PLOTTERY helped us create the ternary plots through code, GUI, and AI. The code was used to compute the data, and the GUI helped us verify our computations. The GUI also allowed us to discover settings for corner shape and visual stylings, all in a live way, without the context-switching of having to look at documentation or query an external AI. Still, the GUI supported only a subset of Matplotlib, inhibiting GUI explorations when the code was not in the supported subset.

Case Study 3 - Code Modality Over Time. Using the log of the participants interactions in PART 2, we estimated where each part of the user's code came from in PART 2—from code, GUI, or AI—by deducing the tokens of the code changed by each interaction. For each task, Fig. 20 plots the proportion of the tokens in the user's currently visible code (excluding

symbols, whitespace, and starter code) that were created (or most recently changed) by each modality as time progresses through each task. It answers the question: if we stopped at time t , what proportions of the user's current cell code as seen at time t came from each modality? Although arguably a good way to interpret the raw data, it is somewhat confusing so the figure was removed from the main paper.

The first author instrumented themselves creating the initial version of this plot and added that record as the "This Plot" line. The plot shows that P_4 , P_7 , P_{10} , and the first author started with AI and AI remained responsible for the majority of their surviving code throughout authoring. P_9 generally preferred more code and GUI editing. Below we describe the creation of Fig. 20 in PLOTTERY.

Before starting to plot Fig. 20, we pasted the first few lines of our input CSV file into a comment so the AI would see the structure of our input data. We then chose to start by building a single subplot first because PLOTTERY's GUI is less helpful when there are multiple subplots—you must change each subplot individually.

We used AI to plot the lines and to add the text labels at the end of each line. We performed code edits so the labels rendered on top of the lines and so they only showed participant ID and not task number. The GUI helped us fine-tune the position of these labels: we set the horizontal and vertical alignment via the Properties panel and directly dragged the labels around on the plot to change their offset. Dragging one moved all of them simultaneously because they came from a single call in a for-loop. We asked the AI to add a stroke to the labels using the "patheffects" library, using this specific vocabulary in our prompt so it would get it right on the first try (it did). The y axis label and title, initially added by the AI, were modified through the GUI. We then explored many variations to see if it were possible to style the lines so they could be individually distinguished. This exploration used all three modalities: AI to scaffold the start of each experiment, code to modify the scaffold (e.g. style per participant instead of per line), and GUI to explore many parameter tweaks. Eventually we decided the story was best told using thicker lines with partial transparency, without trying to distinguish the lines individually.

We asked the AI to abstract the single subplot code into a function and call it three times. This prompt worked, although, because of some unusual bug as mentioned in Sec. V, live feedback became unavailable and the interface lost its fluid feel. Separately, plot elements inside the function were also no longer selectable in the GUI—we have since changed the system AI prompt to avoid this issue (the `ax` input to the function needed a type annotation for MyPy). With degraded GUI support, we tweaked colors and axes limits with code edits, turned off the right and top axes spines through AI, and adjusted some whole-figure parameters through the GUI including the whole-figure title, the figure size, and the resolution (these were outside the function and still GUI-editable).

We instrumented ourselves as we created the plot and added that interaction log to the plot as the "This Plot" line. That final

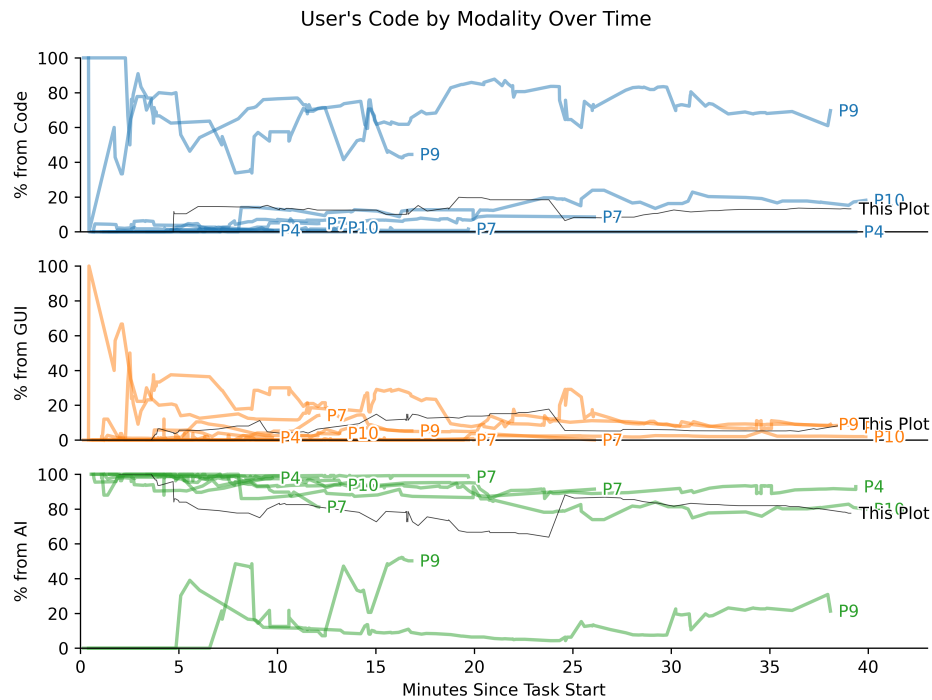


Fig. 20: The provenance of user's code over time for all PART 2 tasks. That is, if we stopped at time t , what proportions of the user's current cell code as seen at time t came from each modality? Additionally, "This Plot" is the initial authoring of this plot by the first author. Most users started with AI and most code comes from AI, but all modalities contribute to authoring.

addition and minor style tweaks afterward are not included in "This Plot". The GUI degradation happened around minute 25.

In Summary: PLOTTERY's three modalities were all useful for constructing Fig. 20. AI helped add the main elements onto the plot, the GUI helped tweak them and explore styling options, and code was used to do what the GUI could not or was too hard to describe to the AI. We did, however, lose some of PLOTTERY's features when we strayed to a code structure that was not encountered in the user studies or prior examples. The degraded GUI and loss of live feedback highlighted the convenience they provide when available.

A Plot Too Complicated for PLOTTERY - Forecast Performance Plot. Fig. 21 is a performance diagram² from an unrelated manuscript in preparation by one of the authors. The plot depicts the probability of detection (POD, *i.e.* recall) and the proportion of true positives (Success Ratio, *i.e.* precision) for two tornado forecasting methods, "SPC" and "NC", at different forecasted probability thresholds (the different colors). Rightward and upward is better. NC's marks have a solid fill when they have a statistically significant difference from the SPC mark at the same threshold (same color). This plot predates PLOTTERY, but here we imagine how PLOTTERY might have—or, rather, might *not* have—helped. The plotting

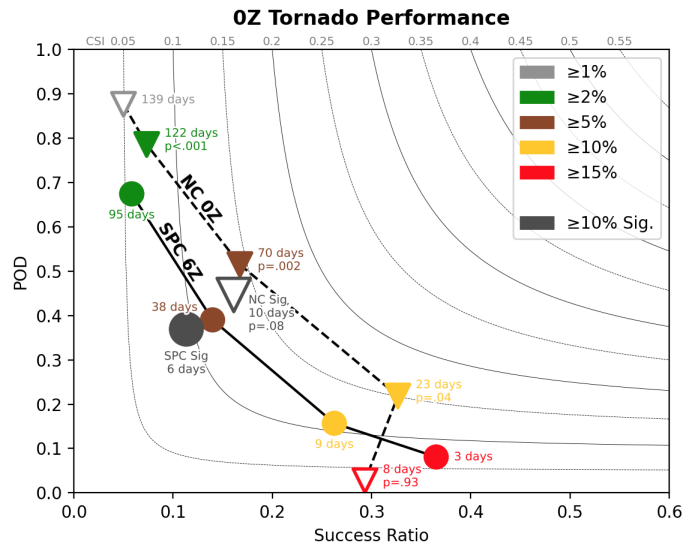


Fig. 21: A severe weather performance diagram.

code was crafted in a function because the manuscript draft itself has 12 such plots, and many more were created over the course of the research project. The function is 163 lines long and has 12 arguments, 12 for-loops and comprehensions, and 26 if-statements, allowing it to handle the many variations of the plot it must make. Only 40 of the 163 lines are Matplotlib calls. PLOTTERY might have helped make a "first draft" of the

²P. J. Roebber, "Visualizing multiple measures of forecast quality," *Weather and Forecasting*, vol. 24, no. 2, pp. 601 - 608, 2009. [Online]. Available: https://journals.ametsoc.org/view/journals/wefo/24/2/2008waf2222159_1.xml

plot, but would become less useful as the function evolved with more special cases for variations. Nevertheless, a few improvements to PLOTTERY might prolong its usefulness had it been used to help author this plot:

- The function has a lot of code to build arrays of properties: *e.g.* an array for the fill colors for a series of marks. An interface to craft simple list comprehensions might be helpful, *e.g.* like Lyra [3]’s simple spreadsheet interface.
- As mentioned in Case Study 2, for-loops to plot a text item or single mark are common. There are 6 such loops for this plot. A way to treat such simple loops as a first-class GUI entity would help—to *e.g.* add filter predicates and simple local computations.
- The items in this legend had to be inserted manually with an impossible-to-remember incantation. GUI support for adding custom legend items would help as well.

C. On-Plot Direct Manipulation

TABLE II: Drag Handle Argument Names

Drag Handle	Arg Name
Shape Body	<i>x and/or y</i>
Top Edge	<i>ymin or top or height</i>
Right Edge	<i>xmax or right or width</i>
Bottom Edge	<i>ymin or bottom or height</i>
Left Edge	<i>xmin or left or width</i>
Legend	<i>loc=(x, y)</i>

Notebook Extension. PLOTTERY consists of a Jupyter Notebook extension, a Python backend, and a Typescript front-end that renders the GUI. The notebook extension installs a Jupyter hook to silently rewrite code before it is sent to the Python backend, replacing calls to `plt.show()` with a PLOTTERY-specific call that tells the Python backend about the variables in scope and the code in the notebook. The call returns a Python object that renders the HTML and Javascript of PLOTTERY’s interface (the object has a `_repr_html_` method for Jupyter).